

Race conditions

A **data race** occurs when:

- Distinct threads **access** a memory location
- At least one of the accesses **modifies** the location
- At least one of the accesses is **non-atomic**
- The accesses are **not ordered** by synchronization

The behaviour of C++ program P on input I is **undefined** if a data race can occur when P is executed on I (the program is allowed to do **anything**).

Compiler Optimization

The compiler optimizes your program **on the assumption that there is no undefined behaviour**. An optimization that assumes no undefined behaviour is: Sound when there is no undefined behaviour (by definition). Sound when there is undefined behaviour, because in that case **anything goes**.

```
static void Foo(int& x) { x = 1; }  
static void Bar(int& x) { while (x==0) {} }  
int main() { int x = 0; // then initialize and join t1 and t2 and return 0 }
```

Keep r in a register – avoid tons of loads from memory.

x is a **non-atomic variable**. If another thread modified x concurrently, that would be a data race. **A data race is an undefined behaviour** (Assume don't happen -> no one else modifies x -> optimize)

```
static void Bar(int& x){int r = x; while (r==0){}}
```

Solution: add volatile will force compiler to load x from memory each time it is accessed. (doesn't solve data race)

Eliminate side-effect free infinite loop.

- This is **an infinite loop without a side effect**
- That's an undefined behaviour (Assume don't happen -> this code doesn't get executed -> optimize)

```
static void Bar(int& x) { // Nothing }
```

Solution: make x atomic so that thread must read the most up-to-date value of x.

A data race occurs even when both threads write the same value.

E.g. When T1 and T2 both store 42 to x. (despite both threads storing 42). When T1 store 42 to x, T2 load from x, and x = 42 originally. (despite existing value of x being 42).